MorphoQL: Declarative Morphological Queries over Multiscale Wavelet Event Relations

An EDGE-THEN-WITHIN Core, Auditable Event Relations, and Recomputable Execution Witnesses

Redha Moulla

AXIA, France

August 2026

Abstract

Morphological queries over time series require an explicit representation of signal changes and a verifiable execution semantics. We present *MorphoQL Core 0.2*, a declarative kernel that compiles `EDGE--THEN--WITHIN` chains over a multi-series relation of signed events extracted from single- or multiscale representations, with provenance down to the numerical coefficients.

Strict satisfaction is separated from scoring, ordering, deduplication, projection, and `LIMIT`. Each visible occurrence is associated with a deterministic-JSON execution witness that binds the occurrence, its rank, the materialized `SELECT` row, the ordered columns, and the fingerprint of the complete projected output. Verification recompiles the query, recomputes strict matches through the declared engine, replays postprocessing and projection, and compares the manifests and the row at the claimed rank. The nested AST schema and the programmatic API enforce the same strict types without silent coercion from booleans, strings, or floating-point values to integers.

Validation comprises 1,200 unique positive queries, 1,200 invalid mutations, 800 differential cases across three execution strategies with no mismatch, and 85 boundary or adversarial tests. On 600 synthetic retail-like series, maximum lines achieve a localized macro F1 of 0.604, versus 0.462 for first differences, 0.587 for a single-scale derivative-of-Gaussian pipeline, and 0.573 for aggregated multiscale maxima. Their overall advantage over the single-scale branch is not established at the 95% level. An observational study on five product–store series is reported as retrospective feasibility rather than causal or operational validation.

Keywords: time series, query language, wavelets, modulus maxima, maximum lines, SQL, temporal joins, provenance.

Contents

1 Introduction

Time-series systems can filter dates, aggregate values, compute windows, and retrieve a subsequence similar to an example. They respond less directly to a compositional request such as:

> "Find an upward transition followed by a downward transition eight to fourteen days later, and then a rebound within the following ten days."

The request does not specify the exact values of the segment. It specifies an event alphabet, an order, and temporal distances. It is therefore closer to a declarative program than to a numerical template.

This general idea has important precedents. The shape-definition language introduced in *Querying Shapes of Histories* represented profiles as symbols such as *up*, *down*, and *stable*, composed through concatenation, choice, and repetition [1]. TimeSearcher, SSTS, shape expressions, temporal logics, and row-pattern engines subsequently proposed other interfaces for querying shapes [5, 11, 17, 6, 7, 20]. MorphoQL therefore claims neither the invention of a shape grammar nor that of sequential operators.

The target problem is narrower: *how can morphological events be materialized, queried, and audited when signal changes must be observed across multiple scales?* The proposed representation is a relation derived from signed maxima and maximum lines in a time–scale plane. A transition becomes a relational event with a timestamp, sign, strength, persistence, scale span, and provenance pointing to the coefficients that produced it. The language then operates on these objects without conflating their geometry with a business cause.

The processing chain is

$$X \xrightarrow{\Phi_\theta} \mathcal{E}_X \xrightarrow{\text{parse}} q \xrightarrow{\text{compile}} P_q \xrightarrow{\text{execute}} \mathcal{M}(q, \mathcal{E}_X) \xrightarrow{\text{witness}} \mathcal{W}. \tag{1}$$

It separates four problems:

1. the **fidelity of the extractor** Φ_θ with respect to the raw signal;
2. the **correctness of the compiler** relative to a given event relation;
3. **provenance**, which connects events to extraction nodes and matches to query constraints;
4. **business or causal interpretation**, which requires variables and assumptions external to the signal.

This work addresses four research questions:

RQ1 Does the `EDGE--THEN--WITHIN` core have an operational syntax and semantics for which three execution strategies return the same strict matches on a multi-series relation?

RQ2 Can a witness connecting a match to the normalized query, configurations, events, manifests, and originating coefficients be materialized and verified by recomputation?

RQ3 How do first differences, a single-scale DoG, aggregated multiscale maxima, and maximum lines perform under identical queries when evaluated as complete pipelines?

RQ4 How sensitive is the maximum-line pipeline to selected linking choices, and what can be inferred from its application to non-injected sales series?

The contributions are:

- a multi-series event relation produced by four extractors, in which every identifier, whether generated or supplied explicitly, must equal the content address of the complete event;
- a normative *MorphoQL Core 0.2*, restricted to chains of `EDGE`, `THEN`, and `WITHIN`, with projection, source, threshold, ordering, and limit executed operationally;
- a strict semantics for signs, gaps, series partitions, and thresholds, compiled to exhaustive enumeration, temporal joins, and SQLite SQL;
- a serializable `ExecutionWitness` and a `verify_witness` procedure that recompute internal invariants and, when the required inputs are supplied, the strict relation, visible output, and projected-row fingerprints;
- a factorial campaign of positive, negative, differential, multi-series, boundary, and adversarial validation;
- an evaluation explicitly framed as a comparison of four complete extraction pipelines, supplemented by a targeted ablation and event-cardinality measurements;
- an exploratory observational study on five real series, without causal claims or an estimate of external morphological accuracy.

2 Related Work and Positioning

2.1 Historical Shape Languages and Query Interfaces

Agrawal et al. proposed a shape-definition language for histories in 1995, with a transition alphabet, regular operators, fuzzy matching, rewriting rules, and indexes [1]. This work is a direct precedent: MorphoQL does not introduce the first compositional syntax for shapes. Its distinguishing feature is the materialization of atoms from multiscale wavelet geometry and the exposure of that geometry through an audited relation.

TimeSearcher supports interactive queries through time boxes, value constraints, and angular constraints on derivatives [5]. SSTS transforms signals into symbolic connotations and then applies regular expressions [11]. ShapeSearch provides a flexible algebra for trendline exploration, input through sketches, natural language, and visual expressions, together with an execution engine and a perceptual score [15]. TSseek approximates series by linear segments and translates regular expressions over trends and value ranges into distributed spatial queries [30]. These approaches show that symbolic expressiveness and indexing can be separated; MorphoQL investigates a different representation based on cross-scale persistence and numerical provenance down to maxima.

2.2 Temporal Logics, Streams, and Row Patterns

Signal Temporal Logic formalizes temporal properties over real-valued signals and provides a quantitative robustness semantics [6, 7]. xSTL and AMT combine logic and timed regular expressions for qualitative and quantitative trace analysis [10]. *Quantitative Regular Expressions* (QREs) describe modular numerical transformations over streams and have been used to express detectors in both the time and wavelet domains; StreamQRE compiles such specifications modularly for streaming data [13, 12]. StreamQL provides an algebra of stream transformations combining relational constructs, dataflow, and temporal operators [14]. MorphoQL differs from these general

languages through an event ontology materialized from time–scale maxima and maximum lines, exposed relationally with coefficient-level provenance.

Within data systems, SASE and `MATCH_RECOGNIZE` recognize ordered event sequences [8, 18]. T-ReX extends `MATCH_RECOGNIZE` with segment variables and an optimizer designed for time-series patterns [20]. MorphoQL can be viewed as a domain layer upstream of such engines: the extractor creates events, and the compiler can then produce joins, an automaton, or a row-pattern expression.

2.3 Symbolic Representations and Subsequence Search

Subsequence search based on Fourier representations, SAX, and the Matrix Profile provides effective solutions for query-by-example, motif discovery, and discord detection [3, 9, 23]. SAXRegEx combines a multivariate symbolic representation, regular expressions, and query expansion supporting duration variation and inter-channel offsets [16]. Shape expressions use regular expressions over parametric shapes with a noise-aware semantics [17]. MorphoQL mainly differs in its unit of indexing: multiscale events rather than complete numerical windows or segments.

2.4 Wavelet Maxima and Maximum Lines

Modulus maxima of a wavelet transform can localize singularities and track their propagation across scales [4]. Ridges of analytic transforms instead characterize oscillatory components and their instantaneous frequencies [21, 22]. The present system uses *modulus-maximum lines* for transitions. The term *ridge* is reserved for non-evaluated oscillatory extensions.

2.5 Natural Language and Time Series

CLaSP, TS-CLIP, and LaSTR align text with series or segments in learned latent spaces [24, 25, 27]. Sonar-TS retrieves candidates before verifying programs on the signal [26]. Grammar of the Wave composes primitives into logical trees and grounds them with vision–language agents [28]; ShapeTalk translates text and sketches into editable constraints [31]. T2SP deterministically transforms series into structured programs of trends, periods, and salient events for language-model reasoning [29]. MorphoQL follows a complementary architecture: natural language may compile into a visible program, but satisfaction of a structured query does not depend on a language model and is executed over an event relation with provenance.

2.6 Precise Positioning

Table 1: Summary positioning. The claimed contribution concerns the interface between a WTMM event relation, compilation, and provenance.

Family	Representation	Composition	Difference from MorphoQL
SDL / SSTS / TSseek	alphabet or segments	regular expressions	no WTMM persistence as a relational attribute
QRE / StreamQRE	quantitative stream transformations	modular composition	general language; no materialized WTMM relation with coefficient provenance
SAXRegEx	multivariate SAX symbols	regular expressions and expansion	symbolic search over segments rather than a time–scale event relation
ShapeSearch	perceptual trends and shapes	algebra, sketch, text	no multiscale event relation with coefficient provenance
STL / xSTL	real-valued signal	timed logic	semantics over the signal rather than an extracted relation
`MATCH_RECOGNIZE` / T-ReX	rows or segments	regular patterns	morphological events must be defined upstream
Shape expressions	parametric shapes	regular expressions	primitive fitting rather than time–scale lines
CLaSP / LaSTR / Sonar-TS / T2SP	latent space, features, or program	text / programs	free-form interface or reasoning without an audited relational WTMM representation
MorphoQL Core 0.2	signed WTMM events	chain and bounded gaps	multiscale provenance and direct relational compilation

The claim is therefore not “a new shape language in general,” but the following:

> MorphoQL investigates an event relation derived from signed maxima and maximum lines as the substrate of a declarative kernel that can be compiled and audited through time–scale provenance.

3 Data Model and Two Levels of Correctness

3.1 Series, Extractor, and Event Relation

Let a corpus of regularly sampled series be

$$X^{(s)} = \big((t_i, x_i)\big)_{i=0}^{T_s-1}, \qquad s \in \mathcal{S}, \qquad t_{i+1} - t_i = \Delta t. \tag{2}$$

A parameterized extractor Φ_θ produces an ordered, partitioned relation:

$$\mathcal{E} = \bigcup_{s \in \mathcal{S}} \Phi_\theta(X^{(s)}). \tag{3}$$

A transition event is the tuple

$$e = (s, \mathrm{id}, \tau, \sigma, \alpha, \rho, a_{\min}, a_{\max}, \chi, \Gamma), \tag{4}$$

where s is the series identifier, id a content-addressed identifier, τ the representative time, $\sigma \in \{-1, +1\}$ the sign, α the strength, ρ the persistence, $[a_{\min}, a_{\max}]$ the scale span,

χ the extractor name, and Γ the ordered sequence of provenance nodes. The identifier is computed as

$$\mathrm{id} = \texttt{evt\_} \,\|\, \mathrm{SHA256}(J(e \setminus \{\mathrm{id}\}))\,, \tag{5}$$

where J is the strict deterministic-JSON serialization of all attributes, including provenance. This invariant is enforced by the public API: an explicit identifier supplied by the caller is accepted only when it equals this address. The identifier is therefore invariant to the insertion of an unrelated event; two exactly identical contents intentionally receive the same address and are rejected by the key constraint rather than hidden under arbitrary names. A node contains at least

$$n = (a, t, c, b, k), \tag{6}$$

where a is scale, t time, c the normalized signed coefficient, b the extraction branch, and k an optional scale index.

The materialized relation is closed by an explicit contract. For every $e \in \mathcal{E}$,

$$\begin{aligned} &(s, \mathrm{id}) \text{ is a unique key and both components are nonempty,} \\ &\tau, \alpha, \rho, a_{\min}, a_{\max} \text{ are finite whenever present,} \\ &\alpha \geq 0, \qquad 0 \leq \rho \leq 1, \qquad 0 \leq a_{\min} \leq a_{\max}, \\ &\Gamma \neq \varnothing, \qquad \mathrm{sign}(c) = \sigma \text{ for every provenance node.} \end{aligned} \tag{7}$$

An `EventRelation` instance is also homogeneous in χ. In materialized Core 0.2, the scale span is mandatory and equals the extrema of Γ. For the four built-in extractors, preprocessing and extraction configurations must exactly equal the registered objects in `src/configuration.py`; this requirement also applies to an empty relation that claims the name of a built-in extractor. An empty external relation may omit a configuration, but it is then assigned the explicit status `empty_unconfigured` and cannot be presented as registered. A materialized third-party extension is accepted as a *declared and versioned* configuration when its `schema_version`, `config_kind`, extractor name, and JSON compatibility are valid. Configuration SHA-256 fingerprints are included in the relation.

The API rejects duplicate keys, explicit identifiers inconsistent with content addressing, duplicate content, `NaN` or infinite values, negative strengths, persistence outside $[0, 1]$, missing provenance, missing scale spans, and mixtures of extractors. Programmatic query parameters are closed as well: both bounds of every `Gap` must be finite and satisfy $0 \leq l \leq u$; `MIN_STRENGTH` is finite; the deduplication tolerance is either `None` or a finite nonnegative real; a projection cannot be empty; and aliases follow the same regular expression as the text parser. Serialization uses `allow_nan=False`, and the application schema rejects silent conversion from a floating-point value or string to an integer. Built-in configurations are compared through deterministic JSON and their fingerprints, not through permissive Python-dictionary equality: payloads `2` and `2.0` remain distinct. All three strict engines apply the same validation even when they receive a raw event collection directly. The domain accepted by the implementation therefore matches the domain of the formal semantics and the primary key of the SQLite plan.

Every match lies within one series partition s; cross-series joins are forbidden by the denotation, the Python engines, and the compiled SQL.

3.2 Two Complementary Provenance Layers

We distinguish:

Extraction provenance
: the sequence Γ of numerical nodes that produced each event;

Query provenance
: the normalized program, its AST, required and observed gaps, threshold, score, postprocessing order, and identifiers of the selected events.

These layers are materialized in an `ExecutionWitness`. Its strict deterministic-JSON serialization and round trip are tested. The object includes the schema version, the implementation version and SHA-256 fingerprint, the engine and strict plan, the fingerprint of the sorted relation, the registered or declared preprocessing and extractor configurations and their fingerprints, and the visible-output policy: total ordering, projection, limit, deduplication tolerance, and quasi-duplicate selection rule. It contains exactly as many events and identifiers as the query contains atoms, and exactly one observed and required gap per `THEN` operator. A match witness requires at least one visible output and a rank within that output.

Four API levels are separated:

- `ExecutionWitness.from_json` performs strict schema deserialization without silent coercion;
- `verify_witness_internal` recomputes the record's internal consistency;
- `verify_witness_against_relation` recompiles the query, verifies the input fingerprint, and recomputes the strict set through the declared engine;
- `verify_witness` reproduces the complete visible pipeline: normative ordering, deduplication, limit, and projection, then compares manifests, the projected row, and the rank.

Caller-supplied lists are objects to be compared; they do not become the source of truth. An externally expected policy may be bound explicitly to distinguish intrinsic consistency from authenticity of a claimed protocol. Built-in configurations come from the same `src/configuration.py` registry consumed by the extractors and protocol.

3.3 Execution Witness, Not an Absolute Proof

Two claims remain distinct:

1. $\mathcal{E} \models q$ means that an event tuple satisfies the syntax and constraints of q;
2. $X^{(s)}$ actually contains a given transition or shape is an empirical claim about the fidelity of Φ_θ.

The engine can establish the first claim relative to the relation. It does not prove the second for a general class of noisy signals. The witness therefore explains *why the algorithm accepted the tuple*, without assigning causal or ontological force to that acceptance. Its verifier detects inconsistencies through recomputation but is neither a digital signature, a proof of prior existence, nor an attestation of the witness producer's identity.

3.4 Morphology, Business Semantics, and Causality

A $+ \to -$ chain describes two opposite transitions. It denotes neither a promotion, nor a stockout, nor a product launch. The following separation is enforced:

$$\text{observable morphology} \neq \text{business label} \neq \text{causal effect.} \tag{8}$$

Price, calendar, or campaign metadata may be joined after retrieval, but they do not alter satisfaction of the core language.

4 Extraction Pipelines and Registered Configuration

This section describes the four pipelines used in the benchmark. They share preprocessing, the query engine, and the evaluation protocol, but have pipeline-specific extraction hyperparameters. The results therefore compare *complete pipelines*; they do not causally isolate the effect of cross-scale linking. Every constant that changes the event relation is defined once in `src/configuration.py`. The script writes strict deterministic-JSON files to `config/`, and the same objects are consumed by extraction, relation validation, witness generation, and the experimental protocol.

4.1 Retrospective Retail Preprocessing

For each daily series $x[0], \dots, x[T-1]$, the weekly profile is estimated over the complete series. Let $d(t) \in \{0, \dots, 6\}$ be the day of the week and $m = \text{median}_t\, x[t]$. We define

$$w_j = \text{median}\{x[t] : d(t) = j\} - m, \qquad x^{\text{des}}[t] = x[t] - w_{d(t)}. \tag{9}$$

The slow component is a centered 63-day rolling median requiring at least 15 observations, extended at the boundaries by the nearest available value:

$$b[t] = \text{Med}_{u \in [t-31, t+31]}\, x^{\text{des}}[u]. \tag{10}$$

The residual and its robust standardization are

$$r[t] = x^{\text{des}}[t] - b[t], \qquad z[t] = \frac{r[t]}{1.4826\ \text{MAD}(r) + 10^{-6}}. \tag{11}$$

This preprocessing is retrospective: the weekly profile uses the complete series and the centered median uses future observations. It must not be interpreted as an online alerting mechanism.

4.2 Derivative-of-Gaussian Coefficients

The scale grid is

$$\mathcal{A} = \{1, 2, 3, 4, 5, 7, 10, 14\} \text{ days}. \tag{12}$$

For each $a \in \mathcal{A}$,

$$D_a[t] = a\, \frac{\partial}{\partial t}(G_a * z)[t], \qquad \widetilde{D}_a[t] = \frac{D_a[t]}{1.4826\ \text{MAD}(D_a) + 10^{-6}}. \tag{13}$$

The implementation uses `scipy.ndimage.gaussian_filter1d`, `order=1`, `nearest` padding, and independent MAD normalization at each scale. The single-scale branch uses $\sigma = 1.5$ days and a minimum peak distance of two days; this exception is encoded explicitly in the registered configuration.

4.3 Signed Maxima, Aggregation, and Linking

At scale a, a positive node is a maximum of $\widetilde{D}_a$ and a negative node is a maximum of $-\widetilde{D}_a$. For the multiscale branches, the minimum distance is

$$\delta_a = \max\left(1, \left\lfloor \frac{a}{2} \right\rfloor\right). \tag{14}$$

The unlinked aggregation sums node strengths by time and sign, smooths the score with a Gaussian of $\sigma = 1.2$ days, extracts events with a minimum distance of two days, and assigns as provenance all same-sign nodes lying within ± 1 day of the final peak.

For maximum lines, nodes are processed separately by sign and increasing scale. An active line may skip at most two scale indices. A node n' extends a line whose last time is t_ℓ when

$$|t(n') - t_\ell| \le \eta(a), \qquad \eta(a) = \max(2, 0.65a + 1), \tag{15}$$

and candidate links are ordered by

$$c(\ell, n') = \frac{|t(n') - t_\ell|}{\eta(a) + 10^{-6}} - 0.03\, s(n'). \tag{16}$$

Numerical ties are broken deterministically by cost, active-line index, and node index. Matching is greedy: it is a reproducible heuristic, not an optimal WTMM tracker.

A line is retained if it covers at least two distinct scales. Its representative time is the strength-weighted average of node times for scales up to three days; otherwise the smallest-scale node is used. Persistence and strength are

$$\rho(L) = \frac{|\{a_i\}|}{|\mathcal{A}|}, \tag{17}$$

$$\alpha(L) = \mathrm{median}_i(s_i)\big(1 + 1.7\rho(L)\big) + 0.15 \max_i s_i - 0.03\, \mathrm{std}_i(t_i). \tag{18}$$

Same-sign lines separated by at most two days are merged. The representative time is taken from the strongest line; the merged strength is its strength plus 0.1 times the sum of secondary strengths; provenance is the ordered duplicate-free union of supporting nodes. For single-scale branches, persistence is conventionally set to 1. It then means "presence on the only configured level," not persistence comparable with a multiscale grid.

Table 2: Registered configurations of the four pipelines.

Pipeline	Scales / distance	Thresholds	Construction and additional constants
First difference	level 0; distance 1 d	node 1.00	extrema of $\Delta z[t]$; singleton provenance; conventional persistence 1
Single-scale DoG	$\sigma = 1.5$ d; distance 2 d	node 1.00	normalized Gaussian derivative; singleton provenance; conventional persistence 1
Multiscale maxima	$\mathcal{A}$; δ_a	node 0.80; event 0.60	temporal sum, smoothing $\sigma = 1.2$ d, event distance 2 d, provenance support ± 1 d
Maximum lines	$\mathcal{A}$; δ_a	node 0.65; event 0.30; min. 2 scales	max. skip 2, tolerance $\max(2, 0.65a + 1)$, bonus 0.03, merge 2 d, secondary weight 0.1

Table 3: Traceability of parameters that determine the relation and visible output.

Family	Source / status	Main values	Role
Preprocessing	`src/configuration.py`; `config/preprocessing_config_v7.json`	63 d window, min. 15, MAD 1.4826, $\epsilon = 10^{-6}$	standardized residual series
Single-scale DoG	`config/extractor_configs_v7.json`	$\sigma = 1.5$, threshold 1, distance 2	single-scale events
WTMM aggregation	same file	grid $\mathcal{A}$, 0.80/0.60, smoothing 1.2, support ± 1	unlinked multiscale maxima
WTMM lines	same file	0.65/0.30, min. 2, skip 2, merge 2, weights 1.7/0.15/0.03/0.1	linking, summarization, and merging
Postprocessing	`config/postprocessing_config_v7.json`	deduplication 2 d, first-result policy, limit after deduplication	visible output and witness
Benchmark	`config/benchmark_protocol_defaults_v7.json`	600/180, 100 quantiles, all candidates, tolerance 3 d, 2,000 bootstrap replicates	calibration and evaluation

4.4 Materialized Relational Schema

Table 4: Schema and invariants of the multi-series event relation.

Attribute	Type	Semantics and invariant
`series_id`	nonempty identifier	series partition; all joins in a match remain within it
`event_id`	nonempty identifier	unique key with `series_id`
`time`	finite real / date	totally orderable representative time
`sign`	$\{-1, +1\}$	downward or upward transition under the branch convention
`strength`	finite real ≥ 0	heuristic strength produced by the extractor
`persistence`	finite real $[0, 1]$	proportion of scales, or convention 1 for a singleton grid
`scale_min/max`	finite reals ≥ 0	exact provenance span, with $a_{\min} \leq a_{\max}$
`extractor_name`	nonempty string	homogeneous within a Core 0.2 `EventRelation`
`provenance`	nonempty list	nodes (a, t, c, b, k) consistent in sign and bounds

On the representative validation series, all four branches produce a nonempty identifier and provenance for every event. The validation also checks numerical domains, key uniqueness, extractor homogeneity, and configuration consistency.

5 MorphoQL Core 0.2

5.1 Normative Scope

Core 0.2 expresses a chain of two or more events, each upward or downward, with a closed temporal window between every consecutive pair. Projection, source relation, a strength threshold, ordering, and limit are part of the operational interface. The `RISE`, `FALL`, `HOLD`, and `OSCILLATE` primitives, negation, and repetition remain non-normative.

```
SELECT SERIES_ID, MATCH_START, MATCH_END, SCORE
FROM sales
MATCH (
    EDGE UP AS u
    THEN EDGE DOWN AS d WITHIN 8d..14d
    THEN EDGE UP AS r WITHIN 5d..10d
)
WHERE MIN_STRENGTH >= 0.5
ORDER BY SCORE DESC
LIMIT 20;
```

5.2 Normative Grammar

```
query       ::= SELECT projection FROM identifier
                MATCH "(" chain ")"
                where_clause? order_clause? limit_clause? ";"?
projection  ::= "*" | field ("," field)*
field       ::= MATCH_START | MATCH_END | SCORE | identifier
chain       ::= edge (THEN edge WITHIN interval)+
edge        ::= EDGE (UP | DOWN) (AS identifier)?
interval    ::= duration ".." duration
duration    ::= number unit
where_clause ::= WHERE MIN_STRENGTH ">=" signed_number
order_clause ::= ORDER BY SCORE DESC
limit_clause ::= LIMIT positive_integer
unit        ::= ms | s | min | h | d | w |
                day | days | j | jour | jours
```

The lexer places long units before their short prefixes and enforces a lexical boundary. All eleven declared units have at least one positive test and are converted to days. Keywords are case-insensitive, but unquoted source names and aliases are case-sensitive: `AS U` must be projected by `SELECT U`, not `SELECT u`. Aliases within a pattern must be unique, and intervals satisfy $0 \le l \le u$.

Normalized durations use the shortest decimal representation that reconstructs the internal floating-point value exactly (`repr(float)`), rather than truncation to twelve significant digits. This choice closes the round trip for `ms`, `s`, `min`, `h`, and day or week units. For each of the eleven units, a factorial validation executes the chain `parse -> execute -> make_witness -> to_json -> from_json -> verify_witness`; a duration requiring more than twelve significant digits is included as a regression test. Exact rational representation would be a possible extension but is not required by the numerical contract of Core 0.2.

5.3 Strict Multi-Series Semantics

Let A_i be the set of events with sign σ_i and strength at least θ. For an interval $I = [l, u]$, define the bounded join

$$J_I(P, A) = \{(p, e) : p \in P,\ e \in A,\ s(e) = s(p),\ e.t > \text{end}(p),\ e.t - \text{end}(p) \in I\}, \tag{19}$$

where $s(\cdot)$ denotes the series and $\mathrm{end}(p)$ the time of the last event in the prefix. For a query with k atoms,

$$[\![q]\!]\varepsilon = J_{I_{k-1}}\big(\cdots J_{I_2}(J_{I_1}(A_1, A_2), A_3)\cdots, A_k\big). \tag{20}$$

This denotation permits overlaps between matches and reuse of an event across different matches, but never reuse within a tuple in non-strict order or across different series. Even when an interval has a lower bound of zero, the constraint $e.t > \mathrm{end}(p)$ remains in force; consecutive atoms therefore cannot be satisfied at the same instant.

5.4 Score, Ordering, Deduplication, and Limit

Strict satisfaction depends only on signs, order, gaps, threshold, and series partition. The score is a heuristic postprocessing function:

$$S_{\text{force}} = \min_i \log(1+\alpha_i) + 0.35\,\frac{1}{k}\sum_i \log(1+\alpha_i), \tag{21}$$

from which a penalty of 0.08 is subtracted according to the normalized distance from the center of each window. The coefficients 0.35 and 0.08 are prototype choices, not theoretical constants.

Without `ORDER BY`, matches are returned under the total order

$$(\texttt{series\_id}, t_1, \ldots, t_k, -S, \mathrm{id}_1, \ldots, \mathrm{id}_k).$$

With `ORDER BY SCORE DESC`, they are sorted by

$$(-S, \texttt{series\_id}, t_1, \ldots, t_k, \mathrm{id}_1, \ldots, \mathrm{id}_k).$$

Content-addressed identifiers provide the final tie-breaker when times and scores are equal; the compiled SQL includes the same columns in `ORDER BY`. Deterministic deduplication is applied after sorting; its tolerance is either absent or finite and nonnegative. `LIMIT`, when present, is applied last. The strict plan covered by the equivalence proposition excludes these operations.

5.5 Projection and Source

The query source must match the name of the `EventRelation` supplied to the engine; a mismatch raises an error. `SELECT *` returns the series identifier, boundaries, score, event identifiers, times, and gaps. The `MATCH_START`, `MATCH_END`, `SCORE`, `SERIES_ID`, and alias fields are materialized; an alias projects the time of the corresponding event.

The programmatic API enforces the same domains as the text parser. A sign is the non-Boolean integer -1 or $+1$; gap bounds, thresholds, and tolerances are finite, non-Boolean `int|float` values. Numeric strings and booleans are never converted silently. This rule prevents a directly constructed program from having semantics different from its textual form.

5.6 Non-Normative Extensions

Interval forms such as `HOLD FOR d`, persistent `RISE/FALL` trends, analytic oscillations, Boolean operators, negation, repetition, and translation from natural language are future extensions. They are neither accepted by the Core 0.2 parser nor included in the experimental results.

6 Compilation and Execution

6.1 Three Strategies for Strict Satisfaction

The LALR parser produces an AST containing the source, projection, sequence of signs, intervals, aliases, threshold, ordering choice, and limit. Three strategies execute the same strict semantics:

1. reference enumeration of all increasing k-tuples of indices;
2. iterative prefix extension through temporal joins;
3. a self-join SQL plan executed by SQLite in differential validation.

For three atoms, the strict plan is conceptually:

```
SELECT e1.series_id, e1.event_id, e2.event_id, e3.event_id
FROM events e1
JOIN events e2
  ON e2.series_id = e1.series_id
 AND e2.time > e1.time
 AND e2.time - e1.time BETWEEN l1 AND u1
JOIN events e3
  ON e3.series_id = e1.series_id
 AND e3.time > e2.time
 AND e3.time - e2.time BETWEEN l2 AND u2
WHERE e1.sign = s1 AND e2.sign = s2 AND e3.sign = s3
  AND e1.strength >= theta
  AND e2.strength >= theta
  AND e3.strength >= theta;
```

Ranking, deduplication, user projection, and `LIMIT` are deliberately applied after this strict plan.

6.2 Equivalence Relative to the Relation

Assumption 1 (Strict-relation contract)**.** *The relation is finite and satisfies the contract of Section 3: every* `event_id` *equals the content address of the complete event;* (`series_id`, `event_id`) *keys are unique and nonempty; times, strengths, and scale spans are finite; strengths are nonnegative; persistence lies in* $[0, 1]$*; provenance is nonempty; configurations are valid; and the relation is partitioned by series. Gaps and the threshold are finite. The three engines use the same closed bounds, apply the threshold to every atom, and do not include scoring, visible ordering, deduplication, projection, or limit in the strict set being compared.*

Proposition 1 (Equivalence of the strict core)**.** *Under the preceding assumption, exhaustive enumeration, temporal-join composition, and self-join SQL return the same set of tuples for every Core 0.2 query and every validated event relation.*

Proof. The enumerator considers all strictly increasing k-tuples from a common series and retains exactly those whose signs, strengths, and gaps satisfy the query. The join plan initializes A_1, the set of satisfying prefixes of length one. Assume that after $i-1$ joins, the obtained prefixes are exactly the satisfying prefixes of length i. Operator J_{I_i} appends all and only later events from the same series, with the required sign and strength, whose gap lies in I_i. The resulting prefixes are therefore exactly the satisfying prefixes of length $i+1$. Induction up to k establishes equality. The SQL plan materializes the same predicates in its self-join clauses, and the composite key identifies selected events unambiguously. □

The proposition is mathematical relative to the stated predicates. Software validation uses common floating-point arithmetic, and the three strategies share the parser, AST, data classes, and several helper functions; they are not three independent reimplementations over the real numbers.

The proposition concerns strict satisfaction. It does not cover extractor fidelity, heuristic scoring, ordering, deduplication, projection, limit, or witnesses. These stages are deterministic and tested separately.

6.3 Timed-Automaton View and Complexity

A chain of k atoms can also be represented by an automaton with $k+1$ states. Transition $i \to i+1$ consumes an event with the expected sign; for $i \geq 1$, a clock reset at the previous event must lie within I_i. This view supports a possible streaming execution, provided the extractor is causal and partial states expire.

Exhaustive enumeration is combinatorial in $|\mathcal{E}|^k$. The ordered plan remains sensitive to the number of prefixes and therefore to selectivity of signs and gaps; its output cost is at least linear in the number of produced matches. A database implementation could exploit an index on (`series_id`, `sign`, `time`), window joins, and cardinality statistics.

6.4 Execution Witness for the Complete Visible Output

For every visible occurrence, `execute_query` produces a projected row and an `ExecutionWitness`. Two manifest levels are bound explicitly: the ordered set of visible occurrences and the ordered set of rows materialized by `SELECT`. The following excerpt summarizes the central generated fields:

```
{
  "schema_version": "morphoql.execution-witness/0.8",
  "implementation_version": "0.8.0",
  "implementation_revision": "supplement-v8",
  "engine_name": "relational",
  "strict_plan": "iterative_temporal_join",
  "normalized_query": "SELECT SERIES_ID, u, d, SCORE ...",
  "query_ast": {"source": "sales", "atoms": [...], ...},
  "event_ids": ["...up...", "...down..."],
  "visible_output_manifest_fingerprint": "<sha256>",
  "projected_output_manifest_fingerprint": "<sha256>",
  "projected_row": {
    "columns": ["SERIES_ID", "u", "d", "SCORE"],
    "values": ["A", 10.0, 12.0, 2.392208616791207]
  },
  "output_rank": 1,
  "events": [...]
}
```

The projected row is serialized as two ordered arrays, `columns` and `values`; column order is therefore part of the contract. The `projected_output_manifest_fingerprint` covers all visible projected rows in their final order. The postprocessing configuration also records the deduplication tolerance, selection policy, total order, limit, and declared projection.

`ExecutionWitness.from_json` validates the exact top-level schema, event schema, projected-row schema, and nested `query_ast` schema. AST signs must be non-Boolean integers, and unknown keys are rejected. Complete verification reparses the query, compares ASTs through deterministic JSON, recomputes strict candidates, replays ordering, deduplication, and `LIMIT`, calls the normative projector on every visible occurrence, and then compares the global projected manifest and the row at the claimed rank. Modifying the projector, a projected value, or column order invalidates the witness.

This procedure establishes consistency by recomputation relative to the supplied relation, code, and configurations. It is neither a cryptographic signature, a proof of

prior existence, nor a guarantee that the event relation is an exhaustive representation of the signal. An interoperable implementation outside Python would require an independently versioned JSON canonicalization specification.

7 Language and Compiler Validation

7.1 Factorial Validation Campaign

Validation is independent of the detection benchmark. The 1,200 positive queries are unique after normalization and cross eleven units, three sources, two to four atoms, four threshold states, ordering present or absent, four limits, aliases absent, complete, or mixed, and four projection families. Projected values and column order are checked. Each negative belongs to one of nine mutation families derived from a positive query; all 1,200 mutations are rejected.

Table 5: Coverage of the Core 0.2 validation for implementation 0.8.0.

Dimension	Levels	Count
Units	ms, s, min, h, d, w, day(s), j, jour(s)	11
Projections	star, boundaries, series, declared aliases	4
Sources	three relation names	3
Pattern aliases	none, complete, mixed	3
Threshold, order, limit	$4 \times 2 \times 4$ states	–
Positive queries	two to four atoms, unique after normalization	1,200
Negative mutations	nine derived families	1,200
Differential cases	strict-pattern dimensions	800
Adversarial tests	relation, AST, API, projection, witness	85

The 85 boundary or adversarial tests cover, among other cases, the ordered projected row, its values, and its global manifest; rejection of a forged row or manifest; rejection of a modified projector; rejection of a JSON sign `1.0` and an unknown AST key; rejection of `Atom(True)`, `Atom(1.0)`, string or Boolean gap bounds, and thresholds or tolerances supplied under a convertible but incorrect type. All tests pass.

7.2 Three-Engine Differential Validation

Eight hundred random cases combine one to three series and zero to nine events per series. The campaign targets the strict set and compares the identifiers returned by the reference enumerator, Python temporal joins, and SQLite self-joins. No mismatch is observed. Projection, ordering, deduplication, and limit are tested separately because they belong to visible postprocessing rather than to the strict-equivalence proposition.

The three engines share the parser, AST, and relation contract. The experiment is therefore a differential validation of execution strategies, not three independent implementations of the entire stack.

7.3 Provenance and Witness Validation

On a representative series, all four pipelines produce events with content-addressed identifiers and nonempty provenance. The witness passes a strict JSON round trip.

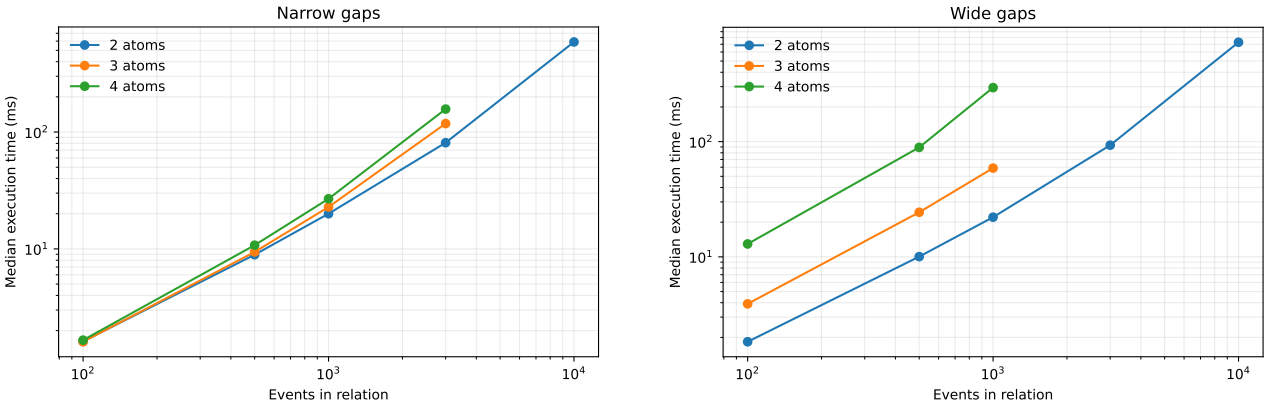
Complete verification recomputes strict candidates, reproduces the visible set, rematerializes SELECT rows, and verifies their fingerprint and the row at the claimed rank. Alterations of a configuration, event, occurrence manifest, projected manifest, projected row, policy, tolerance, or projector are rejected. All eleven time units and a high-precision duration pass the complete chain.

Table 6: Provenance validation on the representative series.

Pipeline	Events	With ID	With provenance	Mean nodes
First difference	105	105	105	1.00
Single-scale DoG	50	50	50	1.00
Multiscale maxima	48	48	48	3.40
Maximum lines	52	52	52	4.15

7.4 Microbenchmark Separated from Correctness Validation

Functional correctness is checked by `run_validation_fast.sh`; the heavier microbenchmark has its own command, `run_microbenchmark.sh`. This separation prevents a continuous-integration check from depending on relations containing 10,000 events. The executed matrix is recorded in `results/compiler_microbenchmark_protocol_v7.json`. For 100, 500, and 1,000 events, patterns of two, three, and four atoms are measured under narrow gaps $[2, 8]$ and wide gaps $[2, 30]$. At 3,000 events, all three pattern lengths are executed for narrow gaps, but only two atoms for wide gaps. At 10,000 events, only two-atom plans are executed in both regimes. The study reports median, P05–P95, and output cardinality. The measurements characterize the Python prototype and are neither a memory benchmark nor a comparison with an industrial DBMS or CEP engine.



(a) Narrow gaps. (b) Wide gaps.

Figure 1: Scaling behavior of the temporal-join plan.

8 Synthetic Benchmark of Extraction Pipelines

8.1 Corpus, Split, and Patterns

Six hundred daily series of 365 points are generated with seeds $91000, \ldots, 91599$. The first 180 are used exclusively to calibrate one threshold per pipeline and query; the remaining 420 form an evaluation split separate from calibration. Noise levels $\sigma \in \{6, 12, 20\}$ are balanced.

Each series combines a level $U(160, 260)$, a slope $U(-0.05, 0.08)$, a weekly sawtooth

$$[-42, -24, -10, 4, 24, 52, -6] \times U(0.7, 1.3), \tag{22}$$

an annual sinusoid with amplitude $U(4, 18)$, a beginning- and end-of-month effect $U(5, 15)$, and AR(1) noise with coefficient $U(0.15, 0.45)$. Each pattern family is selected with probability 0.38, with local exclusion to reduce overlap. Amplitudes lie in $U(45, 95)$, and boundaries may be triangular or trapezoidal.

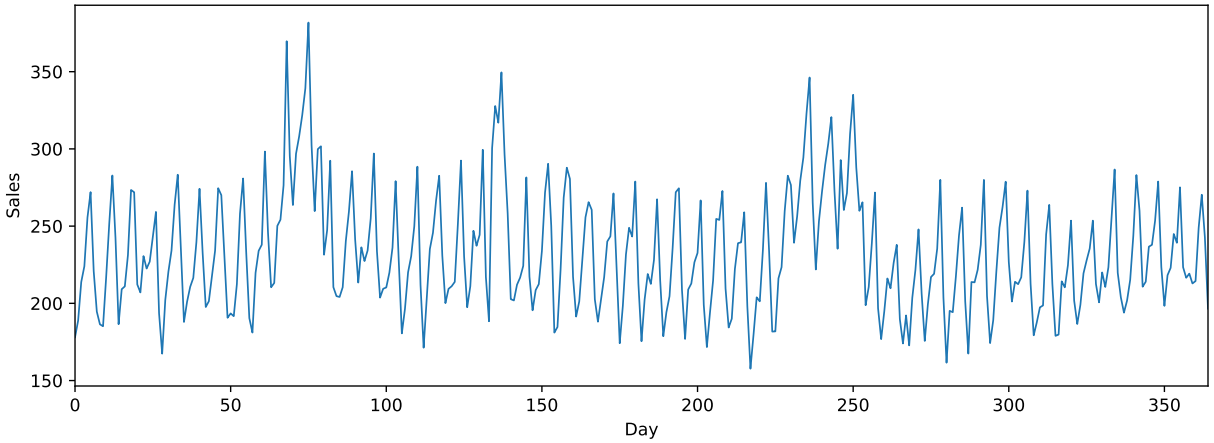


Figure 2: Representative synthetic retail-like series with a weekly sawtooth, slow components, autocorrelated noise, and injected transitions.

Table 7: Evaluated morphological queries.

ID	Expression	Description
Q1	$+ \xrightarrow{1\text{–}4\,\mathrm{d}} -$	brief peak
Q2	$+ \xrightarrow{8\text{–}14\,\mathrm{d}} -$	bounded positive episode
Q3	$+ \xrightarrow{18\text{–}28\,\mathrm{d}} -$	long positive episode
Q4	$- \xrightarrow{5\text{–}10\,\mathrm{d}} +$	temporary trough
Q5	$+ \xrightarrow{8\text{–}14\,\mathrm{d}} - \xrightarrow{5\text{–}10\,\mathrm{d}} +$	rise, fall, rebound
Q6	$+ \xrightarrow{1\text{–}4\,\mathrm{d}} - \xrightarrow{3\text{–}10\,\mathrm{d}} + \xrightarrow{1\text{–}4\,\mathrm{d}} -$	two nearby peaks

8.2 Comparison, Calibration, and Matching

The four pipelines share preprocessing, queries, engine, and evaluation protocol, but differ in their extraction rules. The comparison is therefore between complete pipelines. For each pipeline–query pair, a threshold is selected on the calibration split from 100 quantiles constructed from *all* candidate scores. The criterion is F1, with precision and then recall as tie-breakers. Deduplication uses an L_∞ tolerance of two days before calibration and evaluation and is serialized in the protocol.

A prediction and a ground-truth instance of equal length are compatible when their L_∞ distance is at most three days. Optimal bipartite matching first maximizes the number of compatible pairs and then minimizes total boundary error. The 95% intervals are based on 2,000 paired bootstrap replicates at the series level within the evaluation split, using seed 20260805. They quantify evaluation uncertainty *conditional* on the calibration split and the resulting thresholds; they do not propagate the variability of drawing a new calibration split. Pairwise contrasts are exploratory and not adjusted for multiplicity.

8.3 Artifact Lineage

The distributed candidate tables, extraction configurations, and implementation fingerprints are treated as immutable source artifacts. Metric computation, bootstrap intervals, and output manifests are recorded separately from candidate generation. The language and witness contract does not change the extractors, candidate scores, or benchmark labels. The artifact manifest records whether raw candidate generation was rerun and identifies the exact implementation and configuration fingerprints used at each stage.

8.4 Global Results

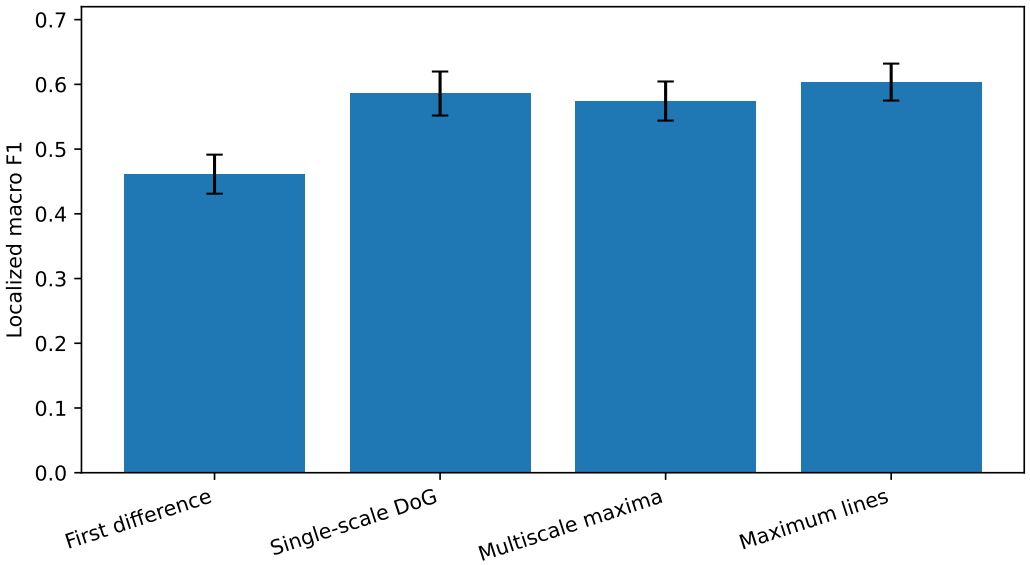


Figure 3: Localized macro F1 on the 420-series evaluation split. The 95% bootstrap intervals are conditional on fixed calibration.

Table 8: Global pipeline performance. Extraction runtimes come from the archived reference run and are reported descriptively; they are not included in statistical comparisons.

Pipeline	Macro P	Macro R	Macro F1 [CI]	Micro F1 [CI]	events/series	ms/series
First difference	.527	.434	.462 [.431;.491]	.489 [.458;.518]	107.0	3.468
Single-scale DoG	.735	.491	.587 [.552;.620]	.604 [.573;.635]	46.4	1.857
Multiscale maxima	.746	.475	.573 [.544;.604]	.592 [.564;.620]	46.1	6.804
Maximum lines	.768	.503	**.604** [.575;.632]	**.627** [.600;.654]	49.9	12.084

The runtime environment and provenance are distributed with the reference artifacts. The values locate the computational cost of the prototype; they do not support a statistical performance comparison or characterize a production database system.

On this split, all three wavelet pipelines have higher point estimates than first differences. The paired macro-F1 gain is $+0.125$ for single-scale DoG, $+0.112$ for aggregated maxima, and $+0.142$ for maximum lines; the conditional intervals exclude zero. Maximum lines have the highest global point estimate. Their gain over DoG is $+0.017$, with 95% CI $[-0.006, 0.040]$, so a global advantage is not established. The maximum-lines versus aggregated-maxima contrast is $+0.030$, CI $[0.012, 0.049]$, on this split, but it is ex-

ploratory, unadjusted, and its point ordering reverses in the additional-seed replication. It does not support a general dominance claim.

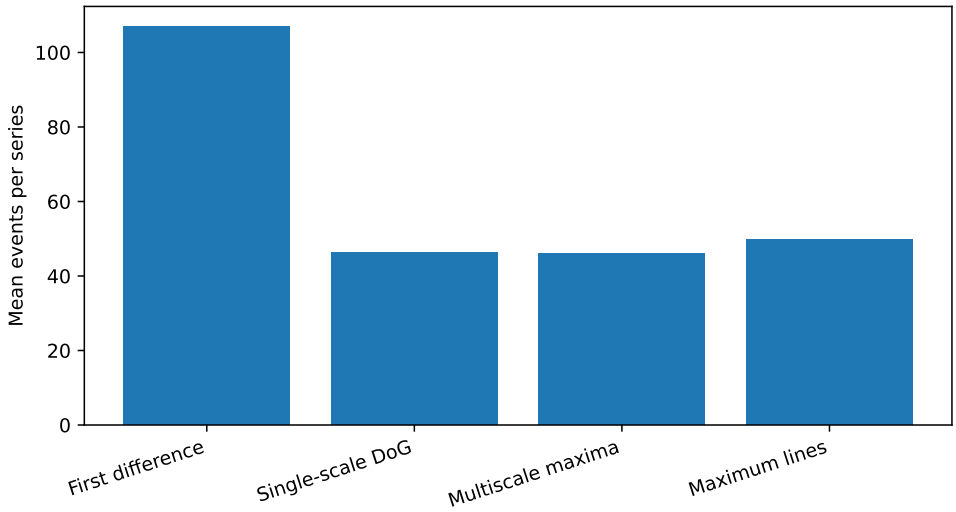


Figure 4: Mean number of materialized events per series before query execution.

8.5 Results by Query and Noise Level

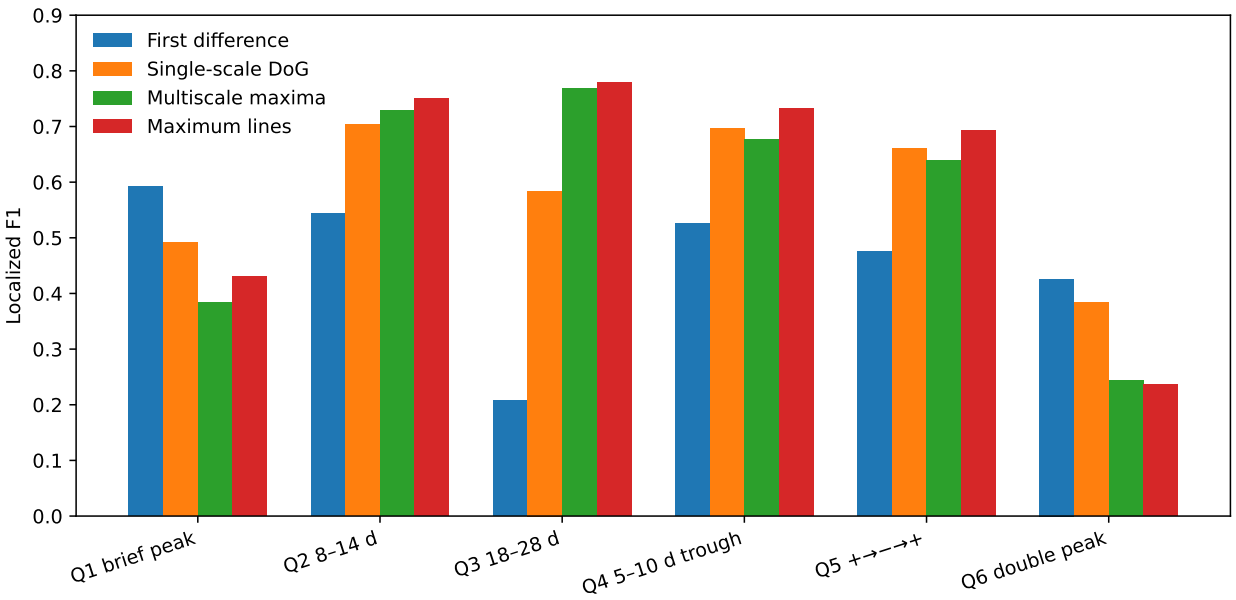


Figure 5: Localized F1 by query and pipeline.

Table 9: F1 by query family.

Query	First diff.	Single DoG	Multi. maxima	Max. lines
Q1 brief peak	**.592**	.492	.384	.430
Q2 8–14 d episode	.544	.703	.728	**.751**
Q3 18–28 d episode	.207	.584	.769	**.779**
Q4 5–10 d trough	.525	.696	.677	**.733**
Q5 $+\to-\to+$	.475	.661	.639	**.693**
Q6 double peak	**.426**	.383	.243	.237

Maximum lines are useful for persistent episodes and Q5. They are unfavorable for brief peaks, whose boundaries merge or disappear when cross-scale persistence is required. A practical policy should select a fine branch for Q1/Q6 and a multiscale branch for more persistent structures; this hybrid policy is not evaluated here.

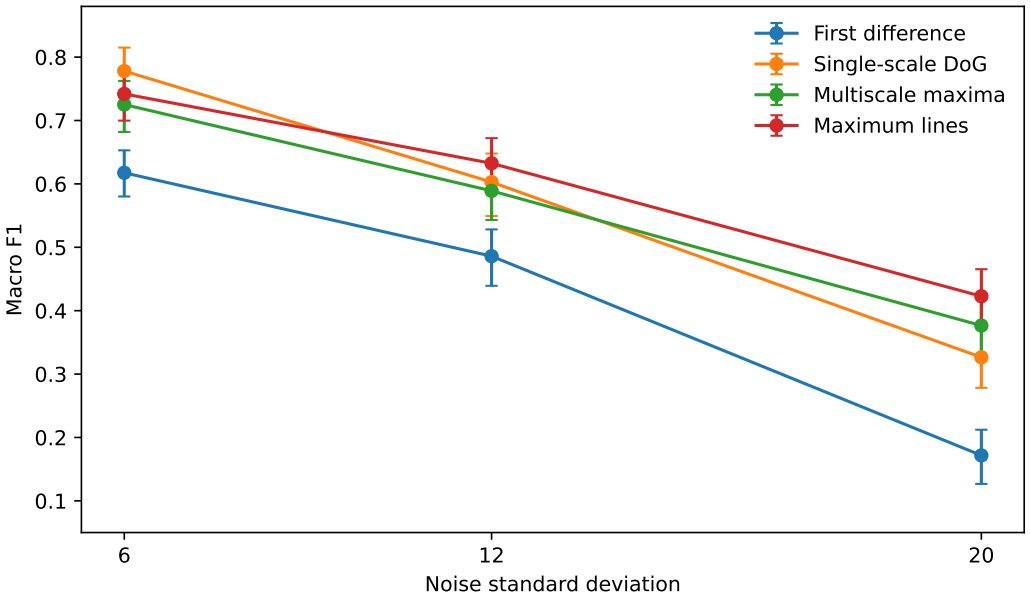


Figure 6: Macro F1 by noise level, with bootstrap intervals.

At $\sigma = 6$, single-scale DoG reaches .778, ahead of maximum lines (.742). At $\sigma = 12$, maximum lines reach .632, ahead of DoG (.603). At $\sigma = 20$, they reach .423, compared with .326 for DoG and .172 for first differences. Cross-scale persistence becomes useful when local extrema are unstable, but remains costly for short events.

8.6 Targeted Ablation of the Maximum-Line Pipeline

A separate experiment uses 300 new series: 90 for calibration and 210 for evaluation. Each variant is calibrated independently on all its candidates. Intervals are based on 2,000 paired bootstrap replicates at the ablation-series level, seed 20260806, conditional on that calibration. They are exploratory and unadjusted for the fifteen pairwise contrasts among variants.

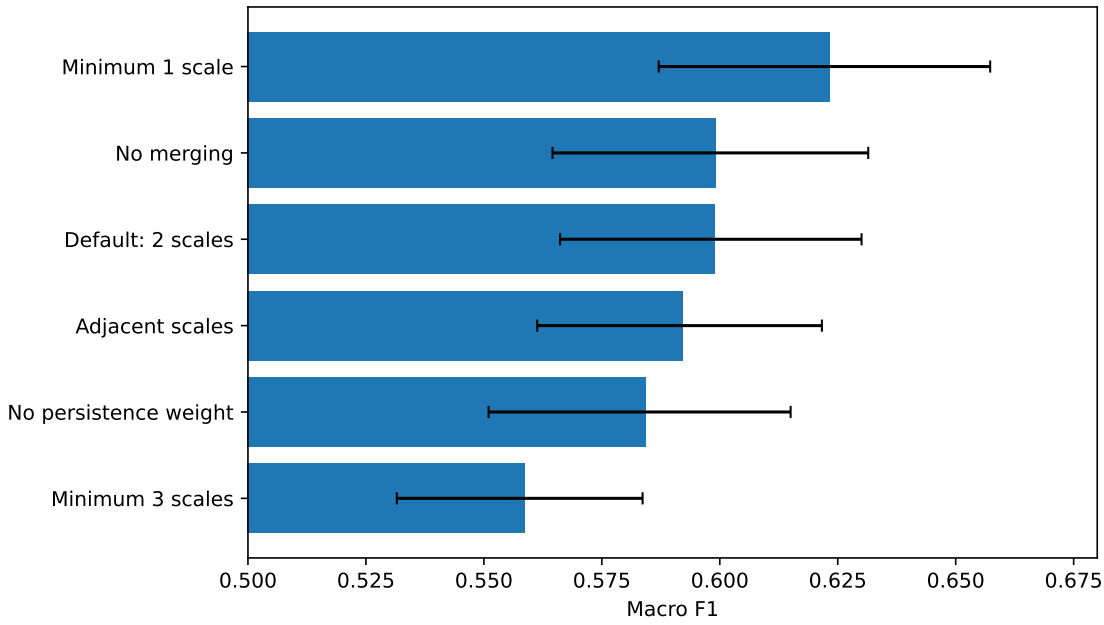


Figure 7: Macro F1 of maximum-line extractor variants on the ablation corpus.

Table 10: Targeted ablation with paired bootstrap intervals.

Variant	Macro P	Macro R	Macro F1 [95% CI]	events/series
Minimum 1 scale	.686	.589	**.623** [.587;.657]	89.1
No merging	.765	.518	.599 [.565;.631]	50.7
Default, minimum 2 scales	.713	.528	.599 [.566;.630]	50.0
Strictly adjacent scales	.701	.523	.592 [.561;.622]	49.7
No persistence weighting	.681	.530	.584 [.551;.615]	50.0
Minimum 3 scales	.681	.483	.559 [.532;.584]	33.8

Allowing a single scale increases F1 by 0.024 relative to the default, with paired interval $[0.0003, 0.048]$, but raises the relation from 50 to 89 events per series and abandons the strong notion of cross-scale persistence. Requiring three scales decreases F1 by 0.040, interval $[-0.062, -0.018]$, and produces a sparser relation. Disabling merging is close to the default in this experiment ($\Delta = 0.00025$, interval $[-0.0076, 0.0076]$). These intervals are exploratory and unadjusted for the fifteen multiple comparisons among six variants. They establish neither optimality nor general superiority; the robust result is the trade-off among recall, relational cardinality, and auditability. The ablation is partial: it does not cover preprocessing, multiple scale grids, or a complete nested hyperparameter selection.

8.7 Replication on Additional Seeds

With code, hyperparameters, split sizes, and calibration procedure fixed, an additional corpus of 120 series is generated from seeds $130000, \ldots, 130119$. The thresholds are not reused from the main benchmark: they are recalibrated under the fixed procedure on 40 new series and evaluated on 80 new series. This is a qualitative replication of the complete procedure under additional seeds, not an application of the original thresholds and not an independent preregistration. Its size does not justify a definitive ranking or new inferential claims.

Table 11: Replication on 80 additional evaluation series.

Pipeline	Macro precision	Macro recall	Macro F1
First difference	.496	.513	.477
Single-scale DoG	.898	.521	.641
Multiscale maxima	.842	.607	**.691**
Maximum lines	.811	.621	.679

The point estimates of all three wavelet branches remain above first differences, but aggregated maxima exceed maximum lines on this split. This reversal confirms that the best representation depends on the corpus. Maximum lines provide explicit cross-scale persistence and coefficient-level provenance. Their event cardinality is comparable to that of the other wavelet pipelines and varies with the persistence criterion, so sparsity should be understood as a tunable trade-off rather than an unconditional advantage.

9 Exploratory Observational Study on Real Sales

9.1 Data and Protocol

The pipeline is applied retrospectively to five daily product–store series from January 2, 2023, to June 7, 2026, corresponding to 1,253 days and 6,265 observations in total. Coverage ranges from 96.6% to 98.9%; missing dates are set to zero under the available historical protocol. Table 12 describes the sample under systematic anonymization, while repeated products and sites are retained to preserve the sample structure.

Table 12: Series included in the observational study.

ID	Product	Store	Coverage	Zeros
S1	Product A (250 g format)	Site 1	98.9%	1.1%
S2	Product A (250 g format)	Site 2	98.4%	1.6%
S3	Product B (250 g format)	Site 1	96.6%	3.4%
S4	Product A (250 g format)	Site 3	96.6%	3.4%
S5	Product B (250 g format)	Site 4	96.7%	3.3%

The same centered 63-day preprocessing is used; the analysis is therefore descriptive and non-causal. Three queries are executed: Q1 (1–4 days), Q2 (8–14 days), and Q3 (18–28 days), with fixed score thresholds 1.358377, 1.409728, and 1.403736, respectively. Prices and promotional markers, available only incompletely, are joined *after* retrieval for exploratory description.

9.2 Descriptive Results

Forty-four matches are obtained and consolidated into 28 unique temporal episodes. Table 13 reports the aggregate results. Detected lift is the ratio of sales within a retrieved window to a local reference level; control lift comes from matched control windows under the internal protocol. The distributed artifacts provide the input schema, a generic aggregation script, de-identified aggregates, and a processing manifest. Proprietary data and row-level annotations are not redistributed. The observational analysis is therefore only partially reproducible rather than fully replayable by a third party.

Table 13: Observational aggregates. “Median promo” is the median across series of the fraction of marked units within windows; it is not the fraction of episodes containing at least one marker.

Query	Matches	Series	Detected lift	Control lift	Excess	Median promo
Q1 (1–4 d)	13	5	1.564	.943	+.737	0.0%
Q2 (8–14 d)	19	5	1.402	.997	+.405	85.5%
Q3 (18–28 d)	12	4	1.004	.982	+.037	6.7%
Unique episodes	28	5	1.392	.984	+.383	0.0%

Among the three series with usable annotations, 18 episodes can be compared with the markers: 7 contain at least one marked unit and 11 contain none. This count is not inconsistent with the zero median for unique episodes because the denominators and aggregation procedures differ. The aggregate median price change is close to zero; the positive $+0.004$ point in the global row is a change in a ratio and does not demonstrate a price reduction.

The windows are selected precisely because they contain salient sales shapes, so a high sales lift is partly tautological. The concentration of markers in Q2 is a descriptive signal compatible with some intermediate-duration episodes, but the sample size, incomplete labels, and dependence among observations preclude characterizing MorphoQL as a general promotion detector.

9.3 Stability and Drift

With thresholds fixed, the observed detection rate increases from approximately 1.05 to 1.71 episodes per series–year–query, a ratio of 1.63; Q2 reaches about 2.22. A second extraction branch corroborates approximately 50% of episodes under the selected agreement window. These values indicate drift in score or event frequency, but corroboration is not independent because the methods share the signal and part of the preprocessing. A production system should monitor this drift and recalibrate using past-only windows.

9.4 Scope of the Study

The study establishes that the pipeline can operate without injected patterns and produce interpretable witnesses. It provides neither blindly annotated morphological ground truth, nor a sufficiently large sample, nor causal control. It therefore cannot estimate real query accuracy or attribute episodes to a commercial intervention.

10 Discussion

10.1 What MorphoQL Contributes

The main contribution is an explicit interface among four layers:

$$\begin{aligned}\text{signal and coefficients} &\longrightarrow \text{multi-series event relation}\\ &\longrightarrow \text{morphological program} \longrightarrow \text{execution witness.}\end{aligned} \tag{23}$$

The language does not depend on the complete coefficient matrix; it depends on an event contract. The relation can be indexed, joined with metadata, and audited. The program can be validated independently of the extractor, while the witness reconnects a match to numerical nodes without conflating traceability with causal truth.

10.2 What the Validation Establishes and Does Not Establish

The 800 differential cases without mismatch support consistency among three execution strategies on the tested core. They prove neither the absence of every software defect, nor extractor fidelity, nor completeness of a broader language. Likewise, the microbenchmark characterizes a direct Python implementation on relations of up to 10,000 events; it does not establish database scalability or competitiveness with a CEP engine.

10.3 Why Maximum Lines Should Not Be the Only Branch

Maximum lines suppress unstable maxima and expose persistence. The benchmark confirms their usefulness under noise and for episodes lasting several weeks. However, this selection removes part of the information carried by brief peaks. The ablation reinforces this diagnosis: reducing the persistence requirement to one scale increases F1 but nearly doubles the number of events. A morphological planner should therefore trade off a fine branch, a multiscale branch, and expected relational cost. Such a policy remains to be evaluated.

10.4 Natural Language and Vector Retrieval

A language model can translate a free-form request into Core 0.2 and allow the parser to validate the resulting program. The decision remains symbolic and auditable. For vague queries not expressible by the core, a maximum-line encoder could produce a compact vector aligned with a pretrained text encoder. A natural hybrid architecture uses hard constraints in MorphoQL and residual similarity in a latent space. This extension is not implemented or evaluated here.

10.5 Morphological Rewriting and Counterfactuals

An explicit query supports controlled rewriting, for example replacing $[8, 14]$ with $[18, 28]$ or reversing a sign. Retrieving observed episodes satisfying the rewritten expression provides morphological analogues. This is shape editing or a morphological counterfactual, not a causal counterfactual for sales.

11 Limitations and Threats to Validity

1. **Narrow core.** Only `EDGE--THEN--WITHIN` chains are normative. Interval, oscillatory, and Boolean primitives remain future work.
2. **Relative correctness.** Equivalence concerns an already extracted finite relation; it does not prove that the events capture every relevant transition in the signal.
3. **Heuristic extractors.** Thresholds, link costs, strength rules, and merging rules are hand-designed. The ablation is targeted rather than a complete nested hyperparameter selection.
4. **Heuristic score.** Score coefficients are not derived from a probabilistic theory; they support ranking and calibration.
5. **Retrospective preprocessing.** The centered median and global weekly profile introduce temporal leakage if the experiment is interpreted as streaming.

6. **Pipeline comparison.** Branches have pipeline-specific hyperparameters and event densities. The benchmark does not isolate the causal effect of multiscale analysis or linking.
7. **Conditional uncertainty.** Main-benchmark intervals condition on the calibration split and thresholds; pairwise contrasts are exploratory and unadjusted for multiplicity.
8. **Synthetic data.** Injected patterns have controlled boundaries and do not cover all irregular sampling, missingness, intermittent demand, or real retail frequencies.
9. **Incomplete baselines.** The evaluation lacks detection comparisons with segmentation, shape expressions, STL/SSTS/T-ReX, the Matrix Profile, and learned models under common annotations.
10. **Witness scope.** Recomputation establishes consistency relative to the relation, implementation, and bound policy; it does not guarantee authenticity of an unanchored policy or identify the producer.
11. **Interoperable canonicalization.** Fingerprints use a deterministic JSON serialization defined by the Python prototype. Cross-language verification requires an independent specification of numbers, Unicode, key ordering, and composite values.
12. **Limited microbenchmark.** Peak memory, index optimization, and comparison with an industrial engine are not reported. At 10,000 events, only two-atom plans are measured.
13. **Small real-data study.** Five product–store series provide neither independent morphological ground truth, adequate statistical power, nor causal validation.
14. **Partial reproducibility of real data.** Proprietary sales and annotations are not redistributed. The schema, generic script, processing steps, and de-identified aggregates are provided, but the private inputs are unavailable.

12 Conclusion

MorphoQL Core 0.2 demonstrates that an event relation derived from time–scale representations can serve as the substrate of a declarative shape-query kernel. Relation-bound verification recompiles a program, recomputes the strict set, reproduces ordering, deduplication, and limit, rematerializes every projected row, and compares column order, values, and the fingerprint of the complete projected output. The nested AST schema and programmatic API enforce a strict type system without silent coercion.

The validation accepts 1,200 unique positive queries, rejects 1,200 derived invalid mutations, passes 85 adversarial cases, and observes no mismatch across 800 differential cases comparing enumeration, Python joins, and SQLite. These tests support the implementation of the evaluated core without replacing a general proof of software correctness, a cryptographic signature of the witness, or validation of extractor fidelity.

On the main synthetic generator, the three wavelet pipelines have point estimates above first differences. Maximum lines achieve the highest global point estimate and are more robust under high noise, but their advantage over single-scale DoG is not established at the 95% level. Main intervals condition on fixed calibration, and pairwise contrasts are not adjusted for multiplicity. The additional-seed replication recalibrates thresholds under the fixed procedure and reverses the point ordering between aggregated maxima and maximum lines. These results motivate a hybrid planner rather than exclusive use of one representation.

The observational study establishes retrospective feasibility, not general morphological accuracy or a promotional or causal interpretation. The next steps are blindly annotated ground truth over a larger collection of real series, causal preprocessing for streaming use, stronger language and detection baselines, relational optimization, and normative extension of the language one primitive at a time.

A Maximum-Line Linking Pseudocode

```
for sign in {+1, -1}:
    active = []
    finished = []
    for scale_index, scale in increasing_scales:
        nodes = maxima[scale_index, sign]
        candidate_links = []
        for line in active:
            if scale_index - line.last_scale_index <= max_scale_skip:
                tolerance = max(2, 0.65 * scale + 1)
                for node in nodes:
                    if abs(node.time - line.last_time) <= tolerance:
                        cost = abs(node.time-line.last_time)/tolerance
                               - 0.03 * node.strength
                        candidate_links.append((cost, line, node))
        greedily assign links by increasing cost
        finish lines absent for more than max_scale_skip indices
        start one line for each unassigned node
    keep lines covering at least min_scales distinct scales
    compute representative time, persistence, strength,
        scale span, and ordered supporting nodes
merge same-sign lines separated by at most merge_days
    while preserving the union of all supporting nodes
assign content-addressed (series_id, event_id)
emit events above the event threshold
```

B Reproducibility and Artifacts

The accompanying artifact provides source code, tests, registered configurations, reference outputs, figures, and the LaTeX source. Correctness validation and load testing are separated:

```
./run_validation_fast.sh
./run_microbenchmark.sh
./run_full_benchmark.sh
PYTHONPATH=src python src/verify_results_v8.py
./build_paper_english.sh
```

By default, scripts write to reproduction directories and do not modify reference outputs. Setting `MORPHOQL_OVERWRITE_REFERENCE=1` enables intentional refresh. The `src/configuration.py` registry generates the configuration JSON files and is the common source for extraction, witnesses, and the experimental protocol.

Table 14: Compliance table for the main reproducible claims.

Claim	Artifact and command	Expected output
Fast validation	`tests/validate_core_v02.py`; `./run_validation_fast.sh`	1,200 positive, 1,200 negative, 85/85 adversarial cases
Differential equivalence	same script	800 cases, 0 mismatches across three engines
Provenance and witness	`results/execution_witness_example_v8.json`; `results/execution_witness_verification_v8.json`	occurrence and SELECT rows reproduced; projector, value, and manifest tampering rejected
Microbenchmark	`tests/run_microbenchmark_v7.py`; `results/compiler_microbenchmark_protocol_v7.json`	exact matrix of executed cells up to 10,000 events
Benchmark and bootstrap	`src/benchmark_morphoql_v7.py`, `src/postprocess_v7.py`	all calibration candidates, 2,000 replicates
Paired ablation	`src/ablation_wtmm_v7.py`	six variants; exploratory unadjusted intervals
Additional-seed replication	`src/additional_seed_replication_v7.py`	40 new calibration series, 80 evaluation series; thresholds recalibrated under the fixed procedure
Numerical verification	`src/verify_results_v8.py`	invariants and numerical results within tolerances, excluding runtimes
English figures	`src/make_figures_english.py`	English PDF and PNG figures

Checksums distinguish inputs, code, and reference outputs. The implementation fingerprint included in each witness is computed over the engine and configuration registry; relation and manifest fingerprints identify the event input and ordered outputs. `verify_witness` checks these claims by recomputation when the required objects are supplied. This does not prove prior existence or the identity of the witness producer. Execution times are not compared through exact equality. For the observational study, the artifacts provide the schema, generic script, and anonymized aggregates, but not proprietary inputs.

C Complete Maximum-Line Results

Table 15: Maximum-line results on the 420-series evaluation split.

Query	TP	FP	FN	Precision	Recall / F1
Q1	157	92	324	.631	.326 / .430
Q2	266	20	156	.930	.630 / .751
Q3	118	17	50	.874	.702 / .779
Q4	290	42	169	.873	.632 / .733
Q5	80	8	63	.909	.559 / .693
Q6	27	42	132	.391	.170 / .237

References

[1] R. Agrawal, G. Psaila, E. L. Wimmers, and M. Zaït. Querying shapes of histories. In *Proceedings of the 21st International Conference on Very Large Data Bases (VLDB)*, pages 502–514, 1995.

[2] J. F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983.

[3] C. Faloutsos, M. Ranganathan, and Y. Manolopoulos. Fast subsequence matching in time-series databases. In *Proceedings of ACM SIGMOD*, pages 419–429, 1994.

[4] S. Mallat and W. L. Hwang. Singularity detection and processing with wavelets. *IEEE Transactions on Information Theory*, 38(2):617–643, 1992.

[5] H. Hochheiser and B. Shneiderman. Dynamic query tools for time series data sets: Timebox widgets for interactive exploration. *Information Visualization*, 3(1):1–18, 2004. doi:10.1057/palgrave.ivs.9500061.

[6] O. Maler and D. Ničković. Monitoring temporal properties of continuous signals. In *FORMATS/FTRTFT*, pages 152–166, 2004. doi:10.1007/978-3-540-30206-3_12.

[7] A. Donzé and O. Maler. Robust satisfaction of temporal logic over real-valued signals. In *FORMATS*, pages 92–106, 2010. doi:10.1007/978-3-642-15297-9_9.

[8] D. Gyllstrom, E. Wu, H.-J. Chae, Y. Diao, P. Stahlberg, and G. Anderson. SASE: Complex event processing over streams. In *CIDR*, 2007.

[9] J. Lin, E. Keogh, L. Wei, and S. Lonardi. Experiencing SAX: a novel symbolic representation of time series. *Data Mining and Knowledge Discovery*, 15:107–144, 2007.

[10] D. Ničković, O. Lebeltel, O. Maler, T. Ferrère, and D. Ulus. AMT 2.0: Qualitative and quantitative trace analysis with extended Signal Temporal Logic. In *TACAS*, pages 303–319, 2018. doi:10.1007/978-3-319-89963-3_18.

[11] J. Rodrigues, D. Folgado, D. Belo, and H. Gamboa. SSTS: A syntactic tool for pattern search on time series. *Information Processing & Management*, 56(1):61–76, 2019. doi:10.1016/j.ipm.2018.09.001.

[12] K. Mamouras, M. Raghothaman, R. Alur, Z. G. Ives, and S. Khanna. StreamQRE: Modular specification and efficient evaluation of quantitative queries over streaming data. In *Proceedings of PLDI*, pages 693–708, 2017. doi:10.1145/3062341.3062369.

[13] H. Abbas, A. Rodionova, E. Bartocci, S. A. Smolka, and R. Grosu. Quantitative regular expressions for arrhythmia detection algorithms. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 16(5):1586–1597, 2019. doi:10.1109/TCBB.2018.2885274.

[14] L. Kong and K. Mamouras. StreamQL: A query language for processing streaming time series. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA), Article 183, 2020. doi:10.1145/3428251.

[15] T. Siddiqui, P. Luh, Z. Wang, K. Karahalios, and A. Parameswaran. ShapeSearch: A Flexible and Efficient System for Shape-based Exploration of Trendlines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 51–65, 2020. doi:10.1145/3318464.3389722.

[16] Y. Yu, T. Becker, L. M. Trinh, and M. Behrisch. SAXRegEx: Multivariate time series pattern search with symbolic representation, regular expression, and query expansion. *Computers & Graphics*, 112:13–21, 2023. doi:10.1016/j.cag.2023.03.002.

[17] D. Ničković, X. Qin, T. Ferrère, C. Mateis, and J. Deshmukh. Specifying and detecting temporal patterns with shape expressions. *International Journal on Software Tools for Technology Transfer*, 23:565–577, 2021. doi:10.1007/s10009-021-00627-x.

[18] D. Petković. Specification of row pattern recognition in the SQL standard and its implementations. *Datenbank-Spektrum*, 22:163–174, 2022.

[19] E. Zhu, S. Huang, and S. Chaudhuri. High-performance row pattern recognition using joins. *Proceedings of the VLDB Endowment*, 16(5):1181–1194, 2023. doi:10.14778/3579075.3579090.

[20] S. Huang, E. Zhu, S. Chaudhuri, and L. Spiegelberg. T-ReX: Optimizing pattern search on time series. In *Proceedings of ACM SIGMOD*, 2023. doi:10.1145/3589275.

[21] J. M. Lilly and S. C. Olhede. On the analytic wavelet transform. *IEEE Transactions on Information Theory*, 56(8):4135–4156, 2010.

[22] J. M. Lilly. Element analysis: a wavelet-based method for analysing time-localized events in noisy time series. *Proceedings of the Royal Society A*, 473:20160776, 2017.

[23] C.-C. M. Yeh et al. Matrix Profile I: All pairs similarity joins for time series. In *IEEE ICDM*, pages 1317–1322, 2016.

[24] A. Ito, K. Dohi, and Y. Kawaguchi. CLaSP: Learning concepts for time-series signals from natural language supervision. arXiv:2411.08397, 2024; revised 2025.

[25] Z. Chen, X. Zhang, and M. Zhu. TS-CLIP: Time Series Understanding by CLIP. In *Proceedings of EMNLP*, pages 4646–4664, 2025.

[26] Z. Tan, Y. Zhao, S. Wang, C. Xu, Y. Liang, X. Liu, S. Pan, and M. Jin. Sonar-TS: Search-Then-Verify Natural Language Querying for Time Series Databases. Accepted at *ICML 2026*; arXiv:2602.17001v3, 2026.

[27] K. Dohi et al. LaSTR: Language-driven time-series segment retrieval. arXiv:2603.00725, 2026.

[28] S. C. Wan, Y. Y. Wang, T. Hou, X. Chang, and A. Jan. Grammar of the Wave: Towards explainable multivariate time series event detection via neuro-symbolic VLM agents. arXiv:2603.11479v3, 2026.

[29] J. Kim, C. Oh, S. Lee, I. Rish, and C. Lee. Representing Time Series as Structured Programs for LLM Reasoning. arXiv:2606.12481v1, 2026.

[30] X. Li, K. Alnuaim, M. Y. Eltabakh, and E. A. Rundensteiner. TSseek: Regular Expression-Based Similarity Search for Distributed Time Series Datasets. arXiv:2606.09824, 2026.

[31] G. Sun, Y. Chen, E. Guo, Y. Yao, and D. Liu. ShapeTalk: Combining natural language and sketch for time-series pattern querying. arXiv:2607.07073, 2026.